

ΛΕΙΤΟΥΡΓΙΕΣ ΠΟΥ ΕΥΤΕΛΟΓΥΝΤΑΙ ΥΠΟ ΣΥΝΘΗΚΗ

- αν μια συνθήκη είναι αληθής, διαυλαδωσθ σε μια εντολή με μια συγκεκριμένη ετικετα
→ **ΕΙΔΑΛΛΩ**, **ΣΥΝΕΧΙΖΕΤΑΙ Η ΑΣΟΛΟΓΙΔΑΥΗ (ΣΕΦΡΑΥΗ) ΕΥΤΕΛΕΣΘ.**

beg rs1, rs2, L1

// αν $(rs1 == rs2)$, διαυλαδωσθ σην εντολή με ετικετα L1.

bne rs1, rs2, L1

// αν $(rs1 \neq rs2)$, τότε διαυλαδωσθ σην εντολή με ετικετα L1

blt rs1, rs2, L1

// αν $rs1 < rs2$, τότε διαυλαδωσθ σην εντολή με ετικετα L1

bge rs1, rs2, L1

// αν $rs1 \geq rs2$, τότε διαυλαδωσθ σην εντολή με ετικετα L1

κώδικας C:

```
if(a==b){  
    x=1;  
}  
else{  
    x=2;  
}
```

κώδικας RISC-V assembly

(a → x10, b → x11, x → x12)

```
bne x10, x11, else_branch  
li x12, 1  
J end-if
```

else_branch:

li x12, 2

end-if:

#code continues here.

Σημετέος προγραμμάτων RISC-V Assembly

.data

.text

.globl main

main:

li a7, 0

ecall

Δομές | Επανάληψης

① while loop

κώδικας C

```
while (a < b) {
```

```
    a = a + 1;
```

```
}
```

κώδικας RISC-V assembly

a → x10, b → x11

```
while_loop:
```

```
bge x10, x11, end_while
```

```
addi x10, x10, 1
```

```
J while_loop #fora-ελεγχουμε
```

```
end_while:
```

#συνεχίζουμε...

② do-while loop

κώδικας C

```
do {
```

```
    a = a + 1;
```

```
} while (a <= b);
```

κώδικας RISC-V assembly

```
a → x10, b → x11
```

```
do-while loop:
```

```
    addi x10, x10, 1
```

```
    blt x10, x11, do-while-loop
```

η λούπη εκτελείται όταν $a \leq b$

④ For loop

κώδικας C

```
for (int i = 0; i < 10; i++) {
```

```
    sum = sum + i;
```

```
}
```

κώδικας RISC-V assembly

```
i → x10, 10 (limit) → x11, sum → x12
```

```
li x10, 0
```

```
li x11, 10
```

```
for-loop:
```

```
    bge x10, x11, end-for
```

```
    add x12, x12, x10
```

```
    addi x10, x10, 1
```

```
    j for-loop
```

```
end-for:
```


Διαδικασίες και συναρτήσεις

Βασικοί μανόινες

① a_0 έως a_7 ($\chi_{10} - \chi_{17}$)

► χρησιμοποιούνται για να περάσουμε ορίσματα στη συνάρτηση.

- Ο a_0 (και ο a_1 αν χρειαστεί) επιστρέφουν το αποτέλεσμα (return value).

② ra (χ_1 - return address)

► αποθηκεύει τη διεύθυνση μνήμης στην οποία πρέπει να επιστρέψει η συνάρτηση μόλις τελειώσει.

③ sp (χ_2 - stack pointer)

► δείχνει στην κορυφή της σσίβας (stack), η οποία χρησιμοποιείται για την αποθήκευση δεδομένων όταν "φέρνουμε" από υποαχρηστή.

Βασικές εντολές * 2 εντολές που ελέγχουν τη ροή των συναρτήσεων *

① jal (Jump and link)

► Καλεί τη συνάρτηση. Αντιγράφει τη διεύθυνση της επόμενης εντολής στον υποαχρηστή ra (για να φέρουμε που θα θυμάμε) και μετά κάνει jump στη συνάρτηση.

② jr (Jump register)

► Επιστρέφει από τη συνάρτηση. Κάνει jump στη διεύθυνση που είναι αποθηκευμένη σε έναν υποαχρηστή.

Παραδείγματα

(D) Χωρίς στίβα

Κώδικας C

```
intadd numbers (int x, int y) {  
    return x+y;  
}
```

```
int main () {  
    int result = add_numbers (5,7);  
}
```

Κώδικας RISC-V assembly

main:

#ορίσματα που μεταχωρούνται σε ao, a1

li ao, 5

li a1, 7

#κλήση της συνάρτησης - το ra αποθηκεύει τη διεύθυνση

#της επόμενης γραμμής

jal ra, add_numbers

#επιστρέφουμε εδώ - το αποτέλεσμα είναι στον ao
mv to, ao

#τερματισμός προγράμματος

li a7, 10

ecall

#ορισμός συνάρτησης

add_numbers:

add ao, ao, a1

#επιστροφή στο main

jr ra.

② ΗΕ @ χρήση σπείβας

► Αν η συνάρτηση A καλέσει τη συνάρτηση B, η τιμή του ra θα αλλάξει.

► Για να μην χάσουμε την αρχική διεύθυνση επιστροφής προς τη main, πρέπει να αποθηκεύσουμε το ra στη σπείβα.

Complex-function:

```
addi sp, sp, -16 } Δεσμεύω χώρο στη σπείβα  
sw ra, 12(sp)    } - αποθηκεύω ra στη σπείβα  
sw s0, 8(sp)
```

```
jal ra, another-function # καλώ τις άλλες συναρτήσεις
```

```
lw s0, 8(sp)  
lw ra, 12(sp)  
addi sp, sp, 16
```

```
ret # επιστρέφω με σωστό ra.
```


KΩΔΙΜΑΣ RISC-V

.data

.text

.globl main

main:

li a7, 10
ecall

Arrays

Εστω $A[12] = h + A[8]$

και η h είναι αναμενόμενη στον

$x21$ και η base address του A

είναι στον $x22$.

• ο πίνακας αποθηκεύεται από
αρχαίους,

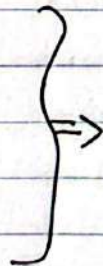
lw $x5, 32(x22)$

add $x6, x21, x5$

sw $x6, 48(x22)$ # αποθηκεύω το
 $x6$ στο $A[12]$ (offset 48)

Δομή Ενδογνώ

if ($C_i == J$) {
 $F = g + h$;
} else {
 $F = g - h$;
}



bne $x28, x29, else_branch$

add $x5, x6, x7$

J end-if

else-branch:

sub $x5, x6, x7$

end-if

f, g, h, i, J

$x5, x6, x7, x29, x29$

Δομή Επανάληψης

while ($save[i] == k$) {
 $i += 1$;
}

$i \rightarrow x22$

$k \rightarrow x24$

$save \rightarrow x25$ (base address)

save πίνακας
αρχαίους 32bit



while-loop:

$x10 = i * 4$

slli $x10, x22, 2$ # byte offset

add $x11, x25, x10$ # διεύθυνση ϵ $save[i]$

lw $x12, 0(x11)$

bne $x12, x24, exit$

addi $x22, x22, 1$

J loop

exit:

ΠΡΟΒΛΗΜΑ ΕΞΕΤΑΣΕΩΝ 1

① Διαβάστε 2 float αριθμούς από τον χρήστη
Αν είναι ίσοι εμφανίστε το μήνυμα 'equal'.

.data

prompt: .string "enter a float\n"

msg: .string "equals\n"

.text

.globl main

main:

print the first prompt

li a7, 4

li a0, prompt

ecall

read first float

li a7, 6

ecall

fmv.s fa1, fa0

print second prompt

li a7, 4

li a0, prompt

ecall

read second float

li a7, 6

ecall # ~~second~~ second float is in a0 now

compare floats

feg.s t1, fa0, fa1 # if t1 == 1 they are equal

beqz t1, exit

equals msg

li a7, 4

la a0, msg

ecall

comparing
floats

ΣΗΜΕΙΩΣΕΙΣ

κλήσιον	κωδ. που α7
print int	1
print float	2
print double	3
print string	4
read int	5
read float	6
read double	7
read string	8

αρχικοποίηση μεταβλητών

li s1,0
li s2,0
li s3,0

② Στο ίδιο πρόγραμμα να διαβάσει συνεχώς από την κονσόλα ακεραίους και να αποθηκεύει 3 μετρητές σε μεταβλητές.

1^{ος}: μετρά αριθμούς που $\in [0 \dots 49]$

2^{ος}: μετρά αριθμούς που $\in [50 \dots 99]$

3^{ος}: μετρά τους υπόλοιπους αριθμούς

• η αναγνώση παύεται όταν εισιχθούν 5 αριθμοί (αυτομάτως, χωρίς) που δεν ανήκουν στο $[0 \dots 99]$.

main:

li s1,0
li s2,0
li s3,0 } αρχικοποίηση μετρητών

read-loop:

li to,5
beg s3,to, print-results } ελέγχος συνθηκών τερματισμού

#Ανάγνωση ακεραίου

```
li a7, 4  
la a0, prompt  
ecall
```

```
li a7, 5  
ecall  
mv t1, a0
```

#Ελέγχος ορίων

```
bltz t1, count-other  
#Ελέγχος αν <=49
```

```
li t2, 49
```

```
ble t1, t2, count-0-49
```

#Ελέγχος αν <=99

```
li t2, 99
```

```
ble t1, t2, count-50-99
```

#άλλη περίπτωση

```
J count-other
```

```
count-0-49 :  
addi s1, s1, 1  
J read-loop
```

```
count-50-99 :  
addi s2, s2, 1  
J read-loop
```

```
count-other :  
addi s3, s3, 1  
J read-loop
```

→ print_results:

```
# s1  
li a7, 4  
la a0, res-m1  
ecall  
li a7, 1  
mv a0, s1  
ecall
```

```
# s2  
li a7, 4  
la a0
```

```
;  
;
```

```
# s3  
;
```

```
exit:  
li a7, 10  
ecall.
```


ΠΡΟΓΡΑΜΜΑΤΙΣΜΟΣ ΣΤΗΝ C

Από διευθυντή και ελεγκτή και αλληλ
βόλε περιλάμβανε (απόσπασμα από το βιβλίο)

③ Να μεταβιβάζονται οι τελικοί τιμές των
μετρητών σε μια συνάρτηση η οποία ετυπώνει
στην οντότητα τις τιμές τους με σχετικά μηνύματα
(της επιλογής σου).

Μετά την εκτέλεση της συνάρτησης, το πρόγραμμα
ολοκληρώνει την εκτέλεσή του ετυπώνοντας το
μήνυμα "Thank you".

① Πίνακας ορισμάτων: οι τιμές των $S1, S2, S3$ θα
ανταγραφούν και καταχωρητές ορισμάτων $a0, a1, a2$
ΠΡΙΝ καλέσουμε τη συνάρτηση.

② Κλήση συνάρτησης: χρησιμοποιούμε την εντολή `jal`
για να πάμε στη συνάρτηση, η οποία αποθηκεύει τη
διεύθυνση επιστροφής που καταχωρητή `ra`.

③ Επιστροφή: Η συνάρτηση θα τελειώνει με την
εντολή `jr ra` για να επιστρέψει στο κύριο πρόγραμμα
(main).

```
;; #κωδικός main από ①, ②
```

```
prep-call: #προετοιμασία ορισμάτων & κλήση συνάρτησης
```

```
mv a0, s1
```

```
mv a1, s2
```

```
mv a2, s3
```

```
jal print_counters #κλήση συνάρτησης - αποθήκευση PC → ra
```

```
#επιστροφή από συνάρτηση + τερματισμός
```

```
li a7, 4
```

```
la a0, thanks-msg
```

```
ecall
```

```
exit:
```

```
li a7, 10
```

```
ecall
```

→ -)

υλοποίηση συνάρτησης

print counters:

mv to, a0

mv t1, a1

mv t2, a2

Επιστροφή πρώτου μέτρητη:

li a7, 4

la a0, res_mrg

ecall

li a7, 1

mv a0, to

ecall

:

:

:

:

Jr ra # Επιστροφή του ενοκλη αμειψιμζα το ja2

ΠΡΟΒΛΗΜΑ ΕΞΕΤΑΣΕΩΝ 2

- α) Ορίστε και ακετιονομίστε ηνάνια Α 10 ακεραιών
β) Γράψτε διαδικασία η οποία θα δέχεται 3 ακεραίους
a, b, c σαν ορίσματα και θα επηρεάζει την τιμή
l αν ισχύει $a < b < c$. Αλλιώς θα επηρεάζει 0.

.data

A: .word 5, 12, 18, 25, 30, 45, 55, 60, 75, 90

.text

.globl main

main:

χρήση δομηματικων τιμων

li a0, 10 # ορίσμα a

li a1, 20 # ορίσμα b

li a2, 30 # ορίσμα c

jal check_order # κλήση της διαδικασίας

μετά την επηρεάση ο μεταχωντής a0 θα ηέρεχε

το αποτέλεσμα 0 ή 1

li a3, 10
e call } *τιμωτισμοί*

check_order:

a < b check

blt a0, a1, check_second_part

j return_false

return_true:

li a0, 1

jr ra # επηρεάση
in main

check_second_part:

b < c check

blt a1, a2, return_true

j return_false

return_false:

li a0, 0

jr ra,